

A SOFTWARE REQUIREMENTS SPECIFICATION

for

BankEase: NEXTGEN NET BANKING

(Account Management, Fund Transfer, Bill Pay & Loan/Investment Services Platform)

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

Abhay Uttam - Roll No: 2400320100027

Under the Guidance of

[Guide Name]

ABES Engineering College

Academic Year 2026-2027

3. FUNCTIONAL REQUIREMENTS

3.1 Authentication and User Management

- FR-01: The system shall allow a new user to register using validated registration data.
- FR-02: The system shall authenticate users through email/password login.
- FR-03: Successful authentication shall return a JWT access token used for protected API requests.
- FR-04: The system shall enforce USER and ADMIN role permissions on protected operations.

3.2 Account Management

- FR-05: An authenticated user shall view all accounts owned by that user.
- FR-06: The system shall expose account number, IFSC, type, status, balance and daily transfer limit.
- FR-07: Authorized users shall be able to create an account through the account API.
- FR-08: Account ownership shall be checked before account-specific operations are performed.

3.3 Beneficiary Management

- FR-09: A user shall add a beneficiary with name, account number, IFSC and related bank information.
- FR-10: A user shall list their beneficiaries.
- FR-11: A user shall deactivate a beneficiary; the operation shall be protected by authentication and ownership validation.

3.4 Fund Transfer

- FR-12: A user shall initiate a transfer using a source account and destination/beneficiary details.
- FR-13: The service shall validate source-account ownership, beneficiary ownership when applicable, account status and transfer amount.
- FR-14: The service shall check available balance and record a unique transfer reference, mode, amount, remarks and status.
- FR-15: Transfer history shall be available to the authenticated user.

3.5 Bills, Loans and Investments

- FR-16: Users shall create bill-payment requests and view bill-payment history.
- FR-17: Users shall submit loan applications and view their application records.
- FR-18: Administrators shall view pending loans and record a loan decision.
- FR-19: Users shall view investment products, create investment records, view their investments and redeem an investment.

3.6 Administration

- FR-20: ADMIN users shall view registered users.
- FR-21: ADMIN users shall update KYC status and enable/disable users.
- FR-22: ADMIN-only endpoints shall reject unauthorized roles.

4. NON-FUNCTIONAL AND SECURITY REQUIREMENTS

Category	Requirement
Performance	Normal academic workloads should receive responsive API responses; database queries should use appropriate indexes and bounded result sets.
Security	Passwords shall never be stored as plain text. Authentication, authorization, input validation and secure token handling are required.
Reliability	Financial state changes shall be processed atomically so that partial balance/transaction updates are not committed.
Usability	The Angular UI shall provide clear labels, validation feedback, status indicators and confirmation/error messages.
Maintainability	Backend code shall retain controller-service-repository separation and DTO-based API contracts.
Scalability	Stateless JWT-based backend services should support later horizontal scaling with appropriate infrastructure.
Portability	The application should run on environments supporting Java 21, MySQL and Angular tooling.

4.1 Authentication and Authorization

- Passwords shall be processed using BCrypt through Spring Security's PasswordEncoder.
- JWT authentication is used for protected API requests; the security filter validates the bearer token before protected controller methods.
- Authentication endpoints are public; administrative endpoints require the ADMIN role; other application endpoints require authentication.
- CORS is configured for the local Angular origin `http://localhost:4200`.

4.2 Data and Transaction Security

- Input DTOs use Jakarta Bean Validation for request validation.
- Ownership checks shall prevent one customer from accessing another customer's account, beneficiary, transfer, loan or investment data.
- Fund-transfer processing shall validate amount and available balance before changing account state.
- Audit logging should capture security-sensitive and financial actions where implemented.

4.3 Error Handling

The API shall return meaningful HTTP status codes and structured error responses. Authentication failures, access-denied conditions, validation errors and business-rule failures should not expose sensitive implementation details.

5. EXTERNAL INTERFACES AND REST API REQUIREMENTS

5.1 User Interface

The Angular application provides routes for login, registration, dashboard, accounts, beneficiaries, transfers, bills, loans, investments and administration. Protected routes are guarded by authentication, and the administration route is additionally guarded by admin authorization.

5.2 REST API

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register user	Public
POST	/api/auth/login	Authenticate and issue token	Public
GET	/api/auth/health	Authentication service health	Public
GET	/api/accounts	List user's accounts	USER
POST	/api/accounts	Create account	Authenticated
GET	/api/accounts/{id}	View account	Authenticated/owner
GET	/api/beneficiaries	List beneficiaries	Authenticated
POST	/api/beneficiaries	Add beneficiary	Authenticated
DELETE	/api/beneficiaries/{id}	Deactivate beneficiary	Authenticated
GET	/api/transfers	Transfer history	Authenticated
POST	/api/transfers	Create transfer	Authenticated
GET	/api/bill-payments	Bill history	Authenticated
POST	/api/bill-payments	Pay bill	Authenticated
GET/POST	/api/loans	List/apply for loans	Authenticated
GET	/api/investments/products	List investment products	Public
GET/POST	/api/investments	List/create investments	Authenticated
POST	/api/investments/{id}/redeem	Redeem investment	Authenticated
GET	/api/admin/users	List users	ADMIN
PATCH	/api/admin/users/{id}/kyc	Update KYC	ADMIN
PATCH	/api/admin/users/{id}/enabled	Enable/disable user	ADMIN
GET	/api/admin/loans/pending	Pending loans	ADMIN
PATCH	/api/admin/loans/{id}/decision	Decide loan	ADMIN

5.3 API Data Format

Requests and responses use JSON. Authentication is sent using the Authorization: Bearer <JWT> header for protected endpoints. IDs are represented as numeric identifiers and financial values use decimal amounts.

6. DATA REQUIREMENTS AND SYSTEM DESIGN

6.1 Core Data Entities

Entity	Important attributes / purpose
User	id, firstName, lastName, email, phoneNumber, password hash, KYC status, enabled, failed-login state, role, timestamps
Role	Role type used for authorization (USER / ADMIN)
BankAccount	accountNumber, IFSC, type, status, balance, dailyTransferLimit, owner, optimistic version
Beneficiary	name, accountNumber, IFSC, bankName, nickname, active flag, owner
FundTransfer	referenceNumber, source account, beneficiary, destination details, mode, amount, remarks, status, timestamp
BillPayment	biller, category, consumer number, amount, account, status and reference
LoanApplication	application number, type, requested amount, tenure, interest rate, EMI, status and decision remarks
InvestmentProduct	product code, name, type, expected annual return, minimum investment, active flag
Investment	reference, product, funding account, principal, current value, status and dates
AuditLog	actor/user, action, affected entity, metadata and timestamp

6.2 Relationships

- One user can own multiple bank accounts and beneficiaries.
- A fund transfer belongs to the authenticated user and references its source account and, where applicable, beneficiary.
- Loans and investments are associated with the user; investments also reference an investment product and funding account.
- Roles determine access to protected administrative operations.

6.3 Three-Tier Architecture

Layer	Implementation in supplied project
Presentation	Angular 20.3 frontend, route guards, forms, reusable UI components and API service
Application / Service	Spring Boot controllers, services, DTOs, validation and Spring Security/JWT
Data	Spring Data JPA repositories, Hibernate entities and MySQL database

6.4 Transaction Integrity

The transfer workflow must verify authenticated ownership, beneficiary validity, account status and sufficient balance before committing the financial change. The account entity also contains a version field, supporting optimistic concurrency control. A transfer reference and status provide traceability.

7. TESTING, ACCEPTANCE, LIMITATIONS AND REFERENCES

7.1 Testing Requirements

ID	Test	Expected result
TC-01	Register with valid data	User is created and authentication response is returned.
TC-02	Login with valid credentials	JWT is returned and protected routes become accessible.
TC-03	Login with invalid credentials	Authentication fails without exposing sensitive details.
TC-04	Access another user's account	Request is rejected by authorization/ownership checks.
TC-05	Add and list beneficiary	Beneficiary is created and visible to its owner.
TC-06	Transfer within balance	Transfer record/reference is created and balance is updated according to service rules.
TC-07	Transfer above balance	Transfer is rejected and invalid balance change is not committed.
TC-08	Create bill payment	Bill-payment record is created and history is available.
TC-09	Submit loan	Loan application is created with an application number/status.
TC-10	Admin loan decision	Only ADMIN can update pending-loan decisions.

7.2 Acceptance Criteria

- All listed Angular routes load without compilation errors after npm install.
- Spring Boot starts successfully with a configured MySQL database.
- Public authentication endpoints are reachable while protected APIs reject unauthenticated requests.
- USER and ADMIN access rules match the defined security requirements.
- Core CRUD and workflow operations return appropriate success/error responses.

7.3 Assumptions and Limitations

- The supplied source archive is the authoritative implementation basis for this SRS; it uses Angular and MySQL.
- The project is an academic prototype and does not connect to real banking settlement networks.
- Real production deployment would require regulatory compliance, stronger operational controls, external integrations, monitoring, backups and disaster recovery.

7.4 Future Enhancements

- Two-factor authentication and stronger device/session security.
- Fraud/anomaly detection and transaction risk scoring.
- Integration with authorized payment/biller services.
- Automated email/SMS/push notifications.
- Detailed loan repayment schedules and investment analytics.
- Containerized deployment, centralized logging, backup automation and disaster recovery.

References

- [1] Supplied BankEase project source archive (Angular frontend, Spring Boot backend and MySQL configuration).
- [2] Supplied BankEase project-report PDF — architecture, requirements, database design, security, testing and future-enhancement sections. [filecite turn1file0 L45-L61](#)
- [3] Spring Boot Reference Documentation.
- [4] Spring Security Reference Documentation.
- [5] MySQL Documentation.
- [6] OWASP Application Security guidance.

6. DATA REQUIREMENTS AND SYSTEM DESIGN

The uploaded BankEase implementation uses User, Role, BankAccount, Beneficiary, FundTransfer, BillPayment, LoanApplication, InvestmentProduct

Entity	Important fields / purpose
User	id, email, password, phone, KYC status, enabled, role
BankAccount	id, account number, IFSC, type, status, balance, daily transfer limit, user
Beneficiary	id, name, account number, IFSC, bank name, nickname, active, user
FundTransfer	id, reference, user, source account, beneficiary, destination, mode, amount, status
BillPayment	id, reference, user, account, biller, category, consumer number, amount, status
LoanApplication	id, application number, user, loan type, amount, tenure, interest, EMI, status
Investment	id, reference, user, product, funding account, principal, current value, status
InvestmentProduct	id, code, name, type, expected return, minimum investment, active
AuditLog	id, user, action, description, IP address, created timestamp

Fig. 6.1: Entity Relationship (ER) Diagram

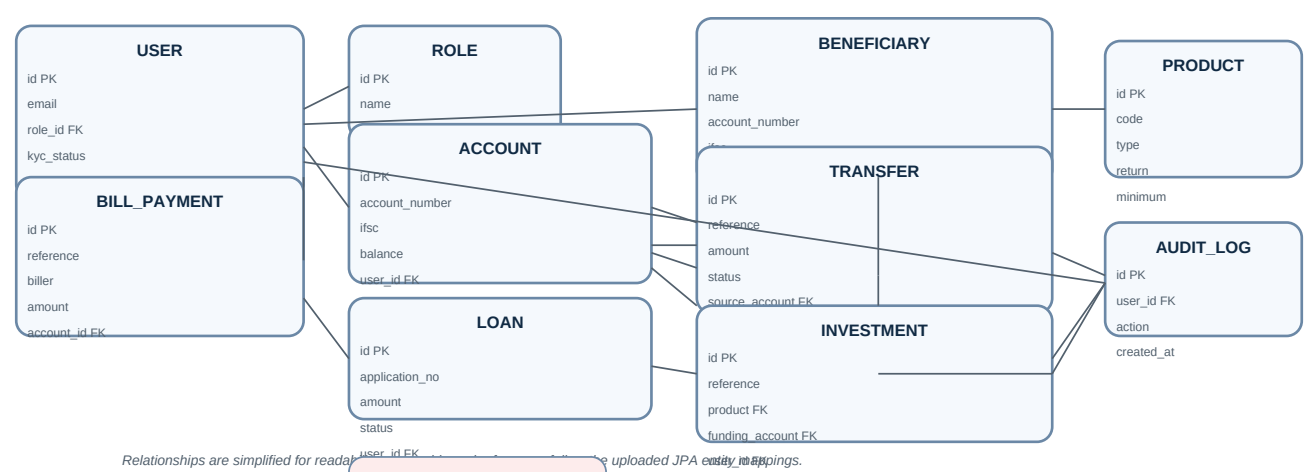
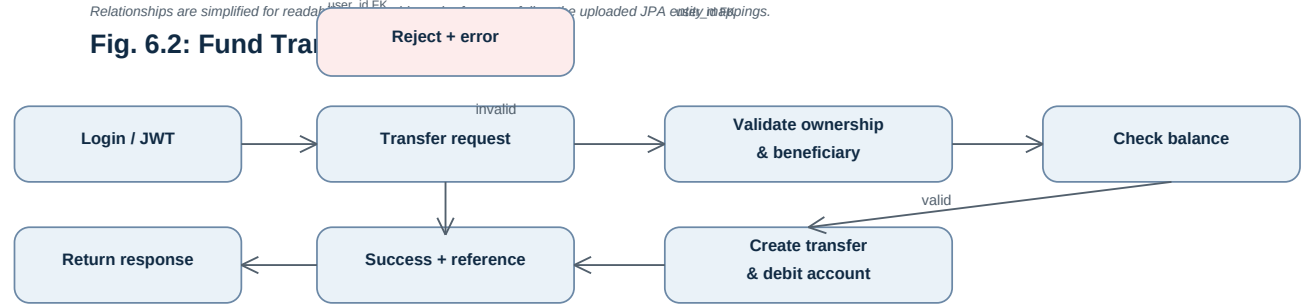


Fig. 6.2: Fund Transfer Flowchart



7. TESTING, ACCEPTANCE, LIMITATIONS AND FUTURE SCOPE

7.1 Testing Requirements

ID	Test	Input / action	Expected result
TC-01	Valid registration	Valid registration	User account created successfully
TC-02	Valid login	Valid login	JWT returned and dashboard accessible
TC-03	Invalid login	Invalid login	Authentication rejected
TC-04	Unauthorized account access	Unauthorized account access	Access denied
TC-05	Add beneficiary	Add beneficiary	Beneficiary saved for owner
TC-06	Valid transfer	Valid transfer	Transfer recorded and balance updated
TC-07	Insufficient balance	Insufficient balance	Transfer rejected without invalid debit
TC-08	Bill payment	Bill payment	Payment record created
TC-09	Loan application	Loan application	Application created with status
TC-10	Admin loan decision	Admin loan decision	Only ADMIN can update decision

7.2 Acceptance Criteria

- Angular frontend builds and starts successfully after dependency installation.
- Spring Boot backend starts with the configured MySQL database.
- Protected endpoints reject unauthenticated requests and ADMIN endpoints reject non-admin users.
- Core account, beneficiary, transfer, bill, loan and investment workflows return expected responses.

7.3 Limitations

- The project is an academic prototype and does not settle real-world funds.
- Production banking integrations, regulatory certification, fraud operations and external payment settlement are outside the supplied project scope.

7.4 Future Enhancements

- Two-factor authentication and stronger device/session security.
- Fraud/anomaly detection and transaction risk scoring.
- Authorized payment/biller integrations and automated notifications.
- Loan repayment schedules, investment analytics, monitoring, backups and disaster recovery.

References

[521] Spring Boot Reference Documentation.

[535] Spring Security Reference Documentation.

[549] MySQL Documentation.

[564] OWASP Application Security guidance.

[578] Supplied BankEase project-report PDF and uploaded BankEase source archive.